

**Angelo Cantoli - Simone Edmondo
Bedini**



POLITECNICO
MILANO 1863

Design Document

Deliverable:	RASD
Title:	Design Document
Authors:	Angelo Cantoli, Simone Edmondo Bedini
Version:	1.0
Date:	07/01/2025
Download page:	https://github.com/angelo-cantoli/CantoliBedini
Copyright:	Copyright © 2025 - Angelo Cantoli, Simone Edmondo Bedini - All rights reserved

Contents

Table of Contents	3
List of Figures	5
List of Tables	5
1 Introduction	6
1.1 Purpose	6
1.2 Scope	6
1.3 Definitions, Acronyms, Abbreviations	6
1.4 Reference Documents	6
1.5 Document Structure	6
2 Architectural Design	8
2.1 Overview	8
2.2 Component View	8
2.2.1 Component Diagram	8
2.2.2 Component Description	8
2.3 Deployment View	10
2.3.1 Deployment Diagram	10
2.3.2 Deployment Description	11
2.4 Component Interfaces	12
2.4.1 User Service Interface	12
2.4.2 Trip Service Interface	13
2.4.3 Path Service Interface	13
2.5 Runtime View	14
2.5.1 Unregistered User Signs Up	14
2.5.2 User Logs in the Account	15
2.5.3 User Manually Adds Bike Trip	16
2.5.4 User Automatically Records Bike Trip	17
2.5.5 User Views their Bike Trip History	18
2.5.6 User views their Bike Trip Statistics	18
2.5.7 User Publishes Recorded Trip	19
2.5.8 User Modifies one of their Saved Trips	20
2.5.9 User Views Possible Bike Routes On Map	21
2.5.10 User Views Additional Information about Selected Path on Map	21
2.5.11 User Starts Navigation on Selected Bike Path	22
2.5.12 User Modifies Profile Data	22
2.5.13 User Logs Out of the Account	23
2.5.14 User Deletes Account	24
2.5.15 User Recovers Password of the Account	25
2.5.16 User Deletes a Recorded Trip	26
2.6 Selected Architectural Styles and Patterns	26
3 User Interface Design	28
3.1 Login and Register Page	29
3.2 Home Page	30
3.3 Search Page	31
3.4 Your Trips Page	32

3.5	Account Page	33
4	Requirements Traceability	34
5	Implementation, Integration and Test Plan	39
5.1	Implementation plan	39
5.2	Integration plan	39
5.3	Test plan	39
6	Effort Spent	41
7	References	42
7.1	Used Tools	42
7.2	Use of Generative AI	42

List of Figures

1	BBP Component Diagram	8
2	BBP Deployment Diagram	10
3	Unregistered User Signs Up Sequence Diagram	14
4	User Logs in the account Sequence Diagram	15
5	User Manually Adds Bike Trip Sequence Diagram	16
6	User Automatically Records Bike Trip Sequence Diagram	17
7	User Views their Bike Trip History Sequence Diagram	18
8	User views their Bike Trip Statistics Sequence Diagram	18
9	User Publishes Recorded Trip Sequence Diagram	19
10	User Modifies one of their Saved Trips Sequence Diagram	20
11	User Views Possible Bike Routes On Map Sequence Diagram	21
12	User Views Additional Information about Selected Path on Map Sequence Diagram	21
13	User Starts Navigation on Selected Bike Path Sequence Diagram	22
14	User Modifies Profile Data Sequence Diagram	22
15	User Logs Out of the Account Sequence Diagram	23
16	User Deletes Account Sequence Diagram	24
17	User Recovers Password of the Account Sequence Diagram	25
18	User Deletes a Recorded Trip Sequence Diagram	26
19	Initial Page	29
20	Login Page	29
21	Register Page	29
22	Home Page	30
23	Search Page	31
24	Route Details	31
25	Your Trips Page	32
26	Trip Details	32
27	Account Page	33

List of Tables

1	Definitions, Acronyms and Abbreviations	6
---	---	---

1 Introduction

1.1 Purpose

BBP is a software designed to simplify the life of everyone who goes on bike trips and wants to enjoy them without worrying about finding the best routes.

It is intended as a community for cyclists, in which every person can share information about their personal routes, so that everyone may take advantage of the information and experience of other bikers. The application is intended for smartphone, where it can make use of GPS and sensor data to seamlessly reconstruct the path a cyclist has taken.

1.2 Scope

The scope of this Design Document (DD) is to define the technical architecture and the detailed design of the Best Bike Paths (BBP) system.

It translates the requirements specified in the RASD into a modular, microservice-based structure, detailing the components, their interfaces, and the user interface that represents the implementation of the system.

It represents the structure to respect in development to ensure that the system is scalable, maintainable, and that it correctly provides a reliable experience for users.

1.3 Definitions, Acronyms, Abbreviations

Acronym	Definition
BBP	Best Bike Paths application
GPS	Global Positioning System
UI	User Interface
SP	Shared Phenomenon
WP	World Phenomenon
UC	Use Case
DA	Domain Assumption
G	Goal

Table 1: Definitions, Acronyms and Abbreviations

1.4 Reference Documents

- Requirements analysis and specification document (RASD) of Best Bike Paths application
- Lecture slides and notes from the course Software Engineering 2 2025/2026

1.5 Document Structure

This document is organized into five main sections, each covering a specific aspect of the BBP (Best Bike Paths) system design:

- **Section 1 - Introduction:** This section provides an overview of the document, defining its purpose and scope. It also includes a list of definitions, acronyms, and abbreviations used throughout the text, along with references to related documents.
- **Section 2 - Architectural Design:** This section describes the system's high-level architecture. It details the microservice-based approach through a Component View and a Deployment View.

Furthermore, it includes a Runtime View where sequence diagrams illustrate how various use cases are executed across the components, and a dedicated subsection for Component Interfaces.

- **Section 3 - User Interface Design:** This section presents the design of the mobile application's interface. It explains the layout and navigation flow, highlighting how the UI has been structured to satisfy the specific needs and goals of the end-users.
- **Section 4 - Requirements Traceability:** This section provides a mapping between the architectural components defined in this document and the functional requirements previously identified in the RASD (Requirements Analysis and Specification Document), ensuring every requirement is addressed by the design.
- **Section 5 - Implementation, Integration, and Test Plan:** The final section outlines the development strategy. It identifies the order in which subcomponents will be implemented and describes the plan for integrating and testing them to ensure system reliability.
- **Section 6 - Effort Spent:** This section provides an overview of the effort spent by each team member during the creation of this document.

2 Architectural Design

2.1 Overview

The BBP system will be developed using a microservice architectural style. This architectural choice decouples the data processing required by certain functionalities of the system (e.g. analyzing data to reconstruct paths or locate potholes, calculating statistics) from the user-interacting functionalities (e.g. account management, trip management). Moreover, this approach reduces dependencies between modules, allows independent deployment, and is more easily scalable. The system will be composed of a client layer (the mobile application), and the microservice based backend. Communication between the two will happen through an API gateway, providing a single entry point. Communication between services will be event-driven (asynchronous) for data processing tasks, to ensure responsiveness, and REST (synchronous) for user data retrieval. Each microservice will manage its own data through a local database, in order to maintain loose coupling between services. The system will also integrate with external services for navigation and weather data retrieval.

2.2 Component View

2.2.1 Component Diagram

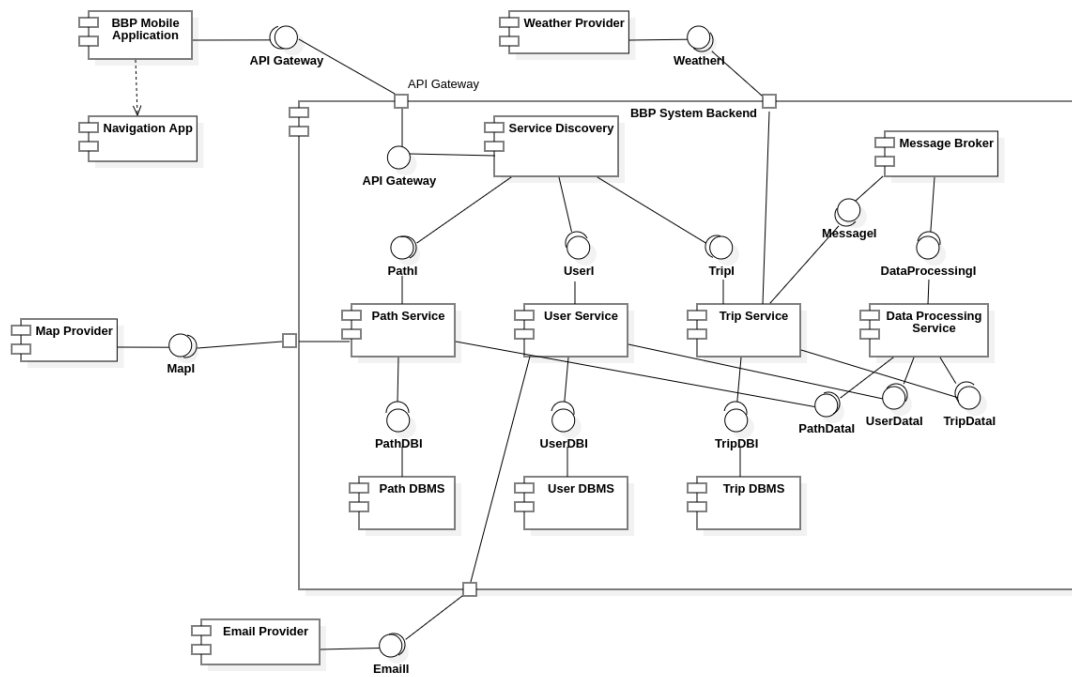


Figure 1: BBP Component Diagram

2.2.2 Component Description

The system is composed of various components, that will be divided into logical layers: the User Interface layer, the Gateway layer, the Application Logic layer, and the Data Layer.

User Interface Layer

- **BBP Mobile Application:** this is the contact point between the system and the users. It is a native mobile application, it hosts the User Interface, and handles all user interaction. It also handles data

acquisition from the mobile device's sensors, and all communication between it and the rest of the system happens through the API Gateway, by way of HTTPS/REST calls.

Gateway Layer

- **API Gateway:** this component acts as the single communication point between the mobile application and the rest of the system. Its purpose is to route all incoming requests to the appropriate microservice, verifying authentication through JSON Web Tokens, and ensuring that traffic is distributed across multiple instances of the various services, in order to properly scale the system based on which services are experiencing a greater load.
- **Service Discovery:** this component maintains a registry of all active microservice instances. When a service, or the API gateway, needs to communicate with another service, they must query this component in order to find the IP address and port of the desired service.

Application Logic Layer (Microservices)

- **User Service:** this service handles all operations relative to a user's scalable account. It manages account creation and deletion, authentication through JWTs, and password recovery.
- **Trip Service:** this service handles the acquisition and storage of trip data. It receives the stream of sensor data during an automated recording, or the inserted data in case of a manually recorded trip, and handles data storage. It will publish an event to the Message Broker upon trip insertion, in order to trigger the asynchronous processing of the received data. It also handles retrieving data for when the user views their personal trip history.
- **Data Processing Service:** this service handles all heavy computation tasks. It is a background service, generally activated by consuming events from the Message Broker. It handles the computation of statistics, the analysis of sensor data for obstacle detection, and the reconstruction of paths. It updates the relevant services after the computation.
- **Path Service:** this service handles the geospatial operations performed on bike paths. It specifically handles the operations required when a user uses the 'path finding' functionality of BBP, wherein the service will search for paths matching the inserted starting and ending locations, and calculate their score.
- **Message Broker:** this component (e.g. Apache Kafka) serves to enable asynchronous communication between services. It is specifically used to decouple the Data Processing Service from the other services, ensuring that high latency computations do not affect system responsiveness.

Data Layer

Since the system is realized following the microservices architecture style, each service will have a distinct, independent database instance. Specifically there will be:

- **User DB:** will store all data relevant to a user's account (user credentials, profile information). Will also store aggregate statistics computed by the Data Processing Service, in order to simplify the handling of this operation. Is accessed exclusively by the User Service.
- **Trip DB:** will store retrieved sensor data and trip summaries. Is accessed exclusively by the Trip Service.
- **Path DB:** a geospatial database (e.g. PostGIS). Will store path geometries.

External Services

- **Map Provider:** an external service (e.g. OpenStreetMap) used to render the map for the UI, and to aid in geospatial computations performed by the backend.
- **Weather Provider:** an external service (e.g. OpenWeatherMap), which is queried in order to retrieve weather conditions along routes, which are used to enrich the trip data.

2.3 Deployment View

The deployment of the BBP system will follow a three-tiered client-server architecture, in line with the microservice architecture style described previously. The system is designed to be hosted in a cloud computing environment (e.g. AWS), which aids in ensuring high availability and scalability, which are critical aspects of the system. The system is divided into the following physical tiers:

- **Client Tier:** consists of the users' mobile devices. These host the BBP mobile application, which is responsible for providing the user interface and collecting all necessary GPS and sensor data.
- **Application Tier:** consists of hosting the application logic within a containerized environment (e.g. kubernetes). Includes the API gateway, the service discovery component, the message broker component, and all the individual microservices.
- **Data Tier:** consists of the various managed databases. To adhere to the microservice architecture style, distinct databases are used for the User, Trip and Path services.

2.3.1 Deployment Diagram

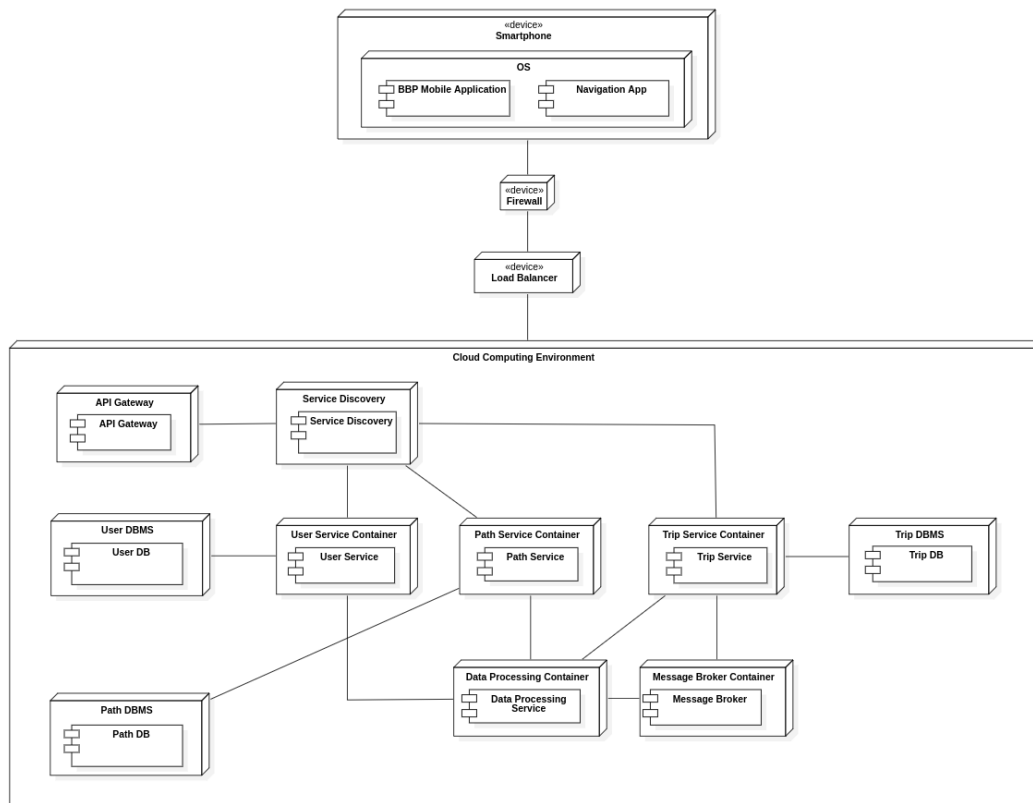


Figure 2: BBP Deployment Diagram

2.3.2 Deployment Description

Containerized Cloud Environment

All the microservices that compose the backend of the application will be deployed in a containerized environment (e.g. Kubernetes), hosted on a cloud provider (e.g. AWS). Each microservice will be packaged as an independent container; this is crucial to ensure scalability (in case of fluctuating loads, multiple instances of a service can be instantiated easily to handle a heavier load, independently from other services with a smaller amount of work), isolation (if a service crashes, the other services will be unaffected). Furthermore, containers ensure that software will run exactly the same way in development, testing and deployment, eliminating consistency issues.

API Gateway

The API gateway component (e.g. Spring Cloud) acts as the single entry point between the public internet and the system. The mobile app can only communicate with the backend through this gateway: all requests are then routed by the gateway to the appropriate internal microservice. This prevents exposing the microservices directly to anything outside of the system, which is crucial for security concerns, and ensures that the mobile application sees the system as a single entity, moving the complexity of service location to the backend.

Message Broker

The message broker (e.g. Apache Kafka) facilitates the communication between services, specifically between the Trip Service and the Data Processing Service. Since the message broker allows asynchronous communication, the Trip Service can send data through it to the Data Processing Service, while sending a confirmation to the corresponding user request. This ensures both that the user has a smoother experience while utilizing the application, since the system hides the computation time by immediately sending acknowledgment, and that in case of a great load (e.g. a lot of users finishing a trip simultaneously), the message broker can act as a buffer while the jobs are processed, so as to not crash the system.

Managed Database per Service

The deployment will include a Managed Database Service (e.g. AWS RDS), which will provide distinct database instances for each microservice. The path service will require a database instance with geospatial extensions (e.g. PostGIS). This enforces the loose coupling required by the microservices architecture, ensuring that services cannot access private data of other services. Furthermore, the single database instances can be specifically optimized for each services' needs.

2.4 Component Interfaces

The component interfaces shown here for the BBP system are the API endpoints for the User Service, Trip Service and Path Service. These all follow the REST constraints and accept and return JSON payloads.

2.4.1 User Service Interface

Base URL: /api/v1/users

URL	Method	Description	Contents/Parameters	Errors
/register	POST	Registers a new user account in the system	Body: {"email": "...", "password": "...", "username": "...", ...}	409 conflict: username/email already taken
/login	POST	Authenticates a registered user and issues a JWT token	Body: {"email": "...", "password": "..."} Response: {"token": "...", "expiration": "..."}	401 unauthorized: invalid credentials
/ {userID}	GET	Retrieves the profile data relative to the specific user ID	JWT authorization token	401 unauthorized: invalid JWT
/ {userID}	PATCH	Updates user profile data	JWT authorization token Body: e.g. {"username": "newUsername"}	401 unauthorized: invalid JWT 409 conflict: username already taken
/ {userID}/stats	GET	Retrieves user’s aggregate statistics	JWT authorization token	401 unauthorized: invalid JWT
/logout	POST	Logs a user out of their account, revoking their JWT authentication token	JWT authentication token	
/ {userID}	DELETE	Deletes a user’s account	JWT authentication token	401 unauthorized: invalid JWT
/reset-request	POST	Begins the password recovery process for a user	Body: {"email": "..."}	400 bad request: no account registered with email
/reset-confirm	POST	Checks the password recovery code and resets the password for the user’s account	Body: {"code": "...", "newPassword": "..."}	400 bad request: invalid code

Table 2: User Service Interface

2.4.2 Trip Service Interface

All of the methods of the Trip Service interface require sending a JWT authorization token, which will be verified by the API gateway to ensure that a user can only handle/create trips in their trip history. In case of an invalid JWT token, the gateway will return a '401 unauthorized' error. For the sake of brevity, these are omitted from the following table.

Base URL: /api/v1/trips

URL	Method	Description	Contents/Parameters	Errors
/	POST	Creates a new trip (manual insertion or initiation of automatic recording)	Body: {"userId": "...", "mode": "MAN/AUTO", ...} Response: {"tripID": "..."} The response contains the ID of the newly created trip.	400 bad request: invalid data from manual insertion
/ {tripID}	PATCH	Updates the state of a trip (e.g. paused, ended)	Body: {"state": "...", ...}	409 conflict: if the trip state change is invalid (e.g. end already ended trip)
/ {tripID} /telemetry	POST	Uploads a batch of GPS and sensor data during the recording of a trip	Body: {...}	
/	GET	Retrieves user's trip history	Body: {"userID": "..."} The body contains the ID of the user whose trip history is requested.	
/ {tripID}	GET	Retrieves data relative to a specific trip		403 forbidden: user doesn't own the trip
/ {tripID}	DELETE	Deletes a trip from the user's trip history		403 forbidden: user doesn't own the trip

Table 3: Trip Service Interface

2.4.3 Path Service Interface

Base URL: /api/v1/paths

URL	Method	Description	Contents/Parameters	Errors
/routes	GET	Retrieves all published paths between two inserted locations	Body: {"start": "...", "end": "..."} Parameters: start, end	
/pathID	GET	Retrieves specific details about a selected path		
/pathID/export	GET	Retrieves data in order to export selected path to navigation app	Body: {"format": "..."} Parameters: pathID, format	

Table 4: Path Service Interface

2.5 Runtime View

2.5.1 Unregistered User Signs Up

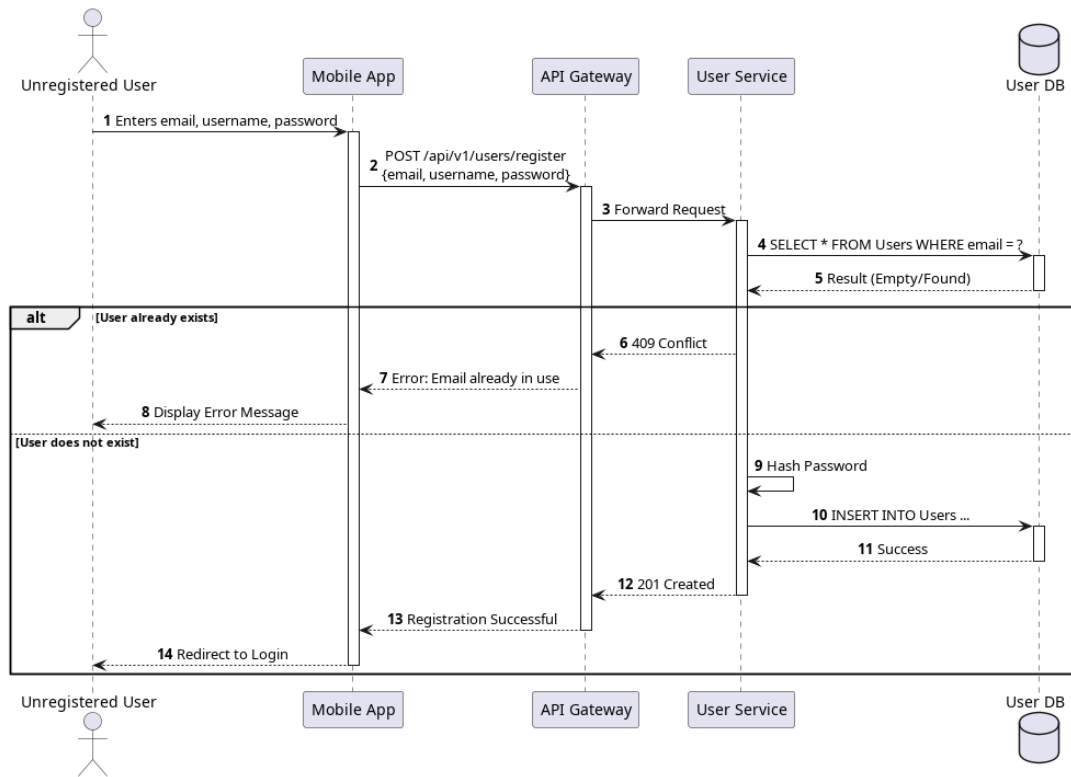


Figure 3: Unregistered User Signs Up Sequence Diagram

When a user registers a new account through the BBP mobile application, it will send the inserted credentials to the API gateway via a POST request. The gateway will forward the request to the User Service, which will handle both input validation and password hashing; the service will check if there are any existing accounts registered with the new user's email, in which case an error will be returned, otherwise the service will hash the password and store the credentials in the User DB.

2.5.2 User Logs in the Account

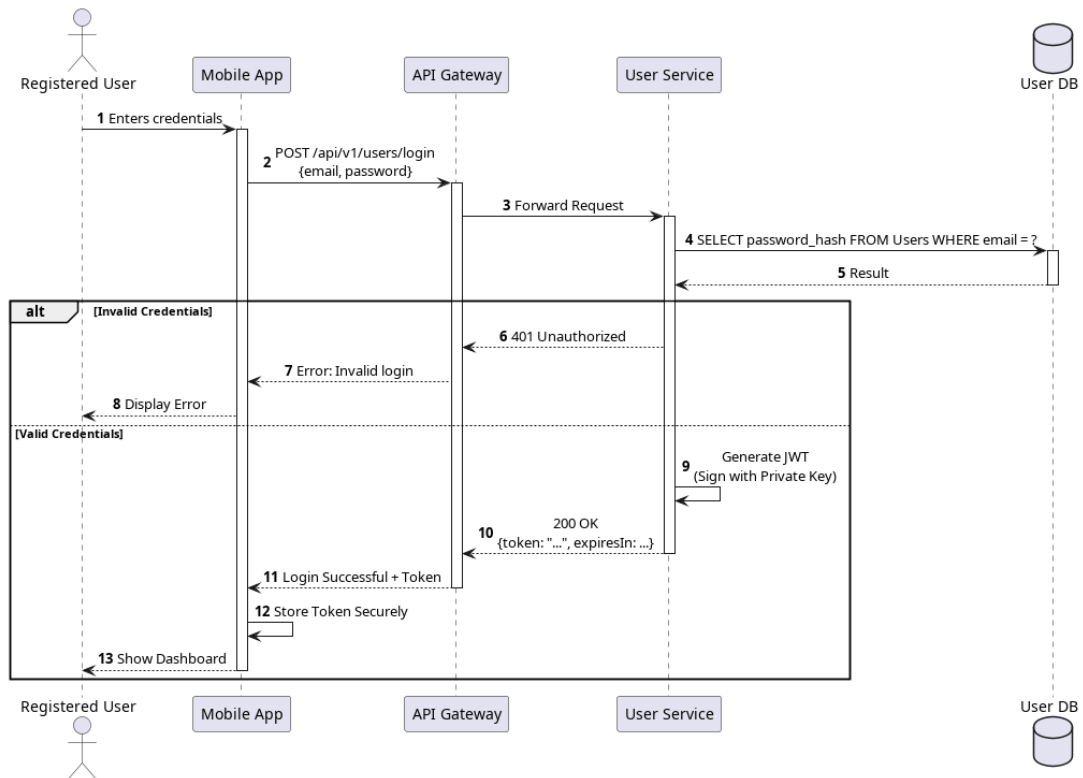


Figure 4: User Logs in the account Sequence Diagram

When a user logs into their account using the BBP application, it will send the inserted credentials to the API gateway via a POST request. The gateway will forward the request to the User Service, which will verify them by querying the User DB. In case of valid credentials, the service will generate a signed JWT (JSON Web Token), which will be stored in the client and attached to all subsequent API requests where authentication is required.

2.5.3 User Manually Adds Bike Trip

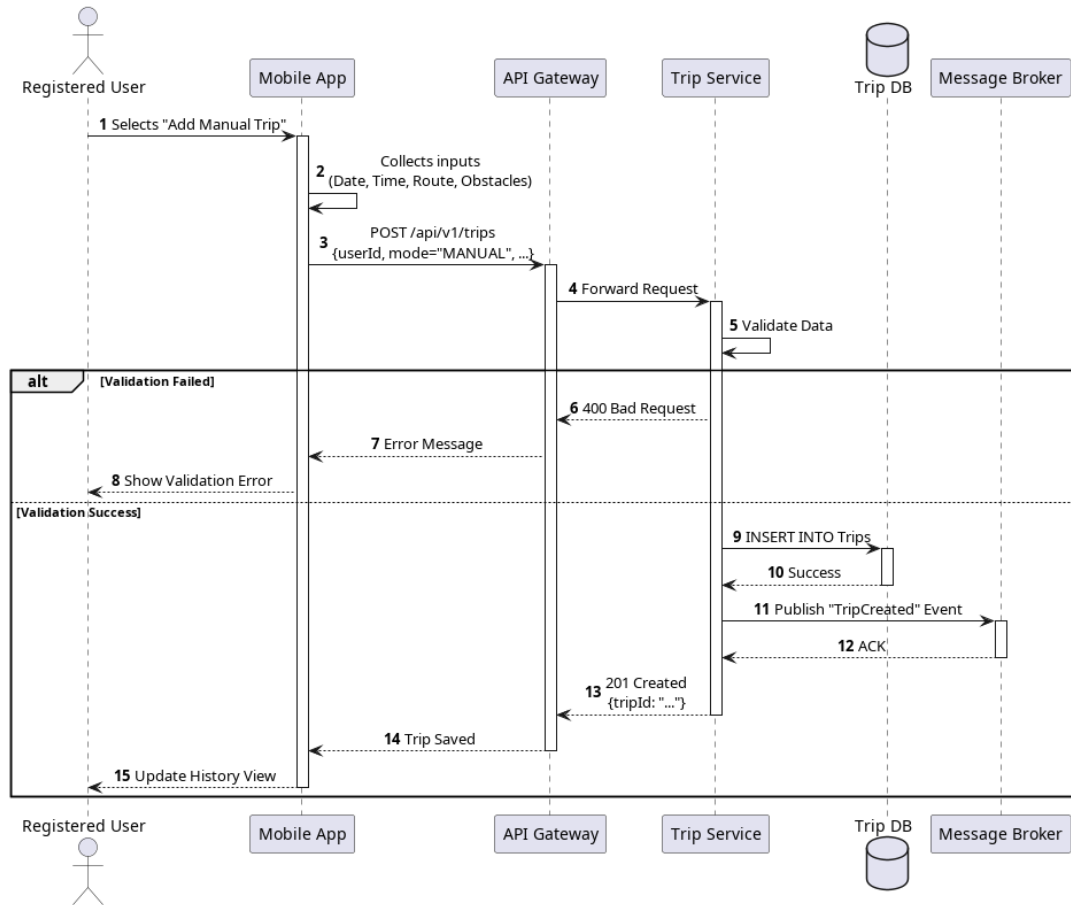


Figure 5: User Manually Adds Bike Trip Sequence Diagram

When a user manually inputs a trip, the mobile app sends all inserted trip data to the API gateway via a POST request, which is then forwarded to the Trip Service. The service will then store the new trip in the Trip DB, and publish an event to the Message Broker to ensure that the Data Processing Service is informed of the insertion of a new trip, and can update the user's aggregate statistics asynchronously, without blocking the response to the user.

2.5.4 User Automatically Records Bike Trip

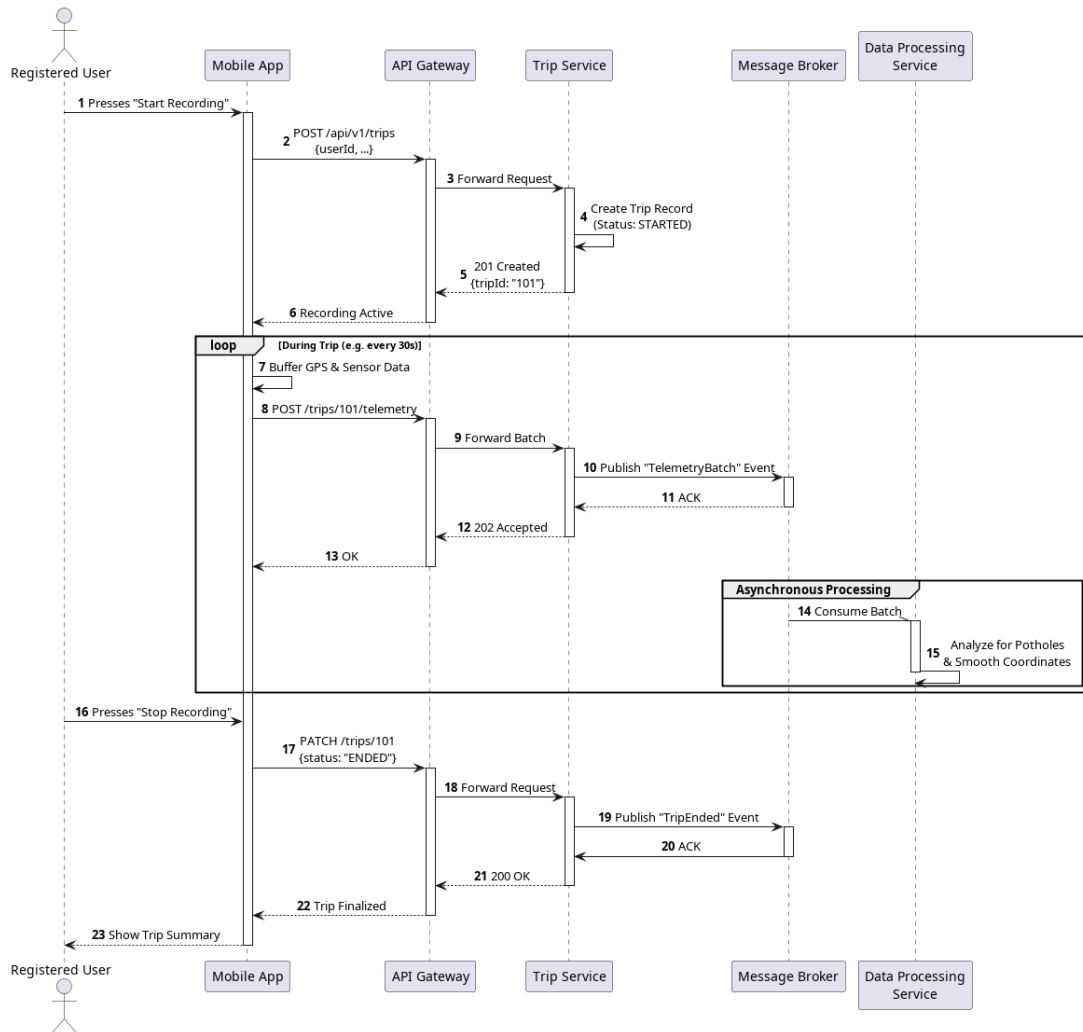


Figure 6: User Automatically Records Bike Trip Sequence Diagram

When a user starts an automatic recording of a trip, the BBP application will send a POST request to the API gateway, which will then forward the request to the Trip Service. The gateway will then notify the app that the system is ready to start the recording process. The application will then collect GPS and sensor data, and periodically send batches of data to the gateway, which are then forwarded to the Trip Service. The service will then publish events to the Message Broker, so that the Data Processing Service may asynchronously process the data while the mobile app immediately receives a response. This ensures that the data processing does not interfere with the recording process. Once the user terminates the recording, the API gateway is notified and the request is forwarded once more to the Trip Service, which publishes a 'TripEnded' event to the Message Broker.

2.5.5 User Views their Bike Trip History

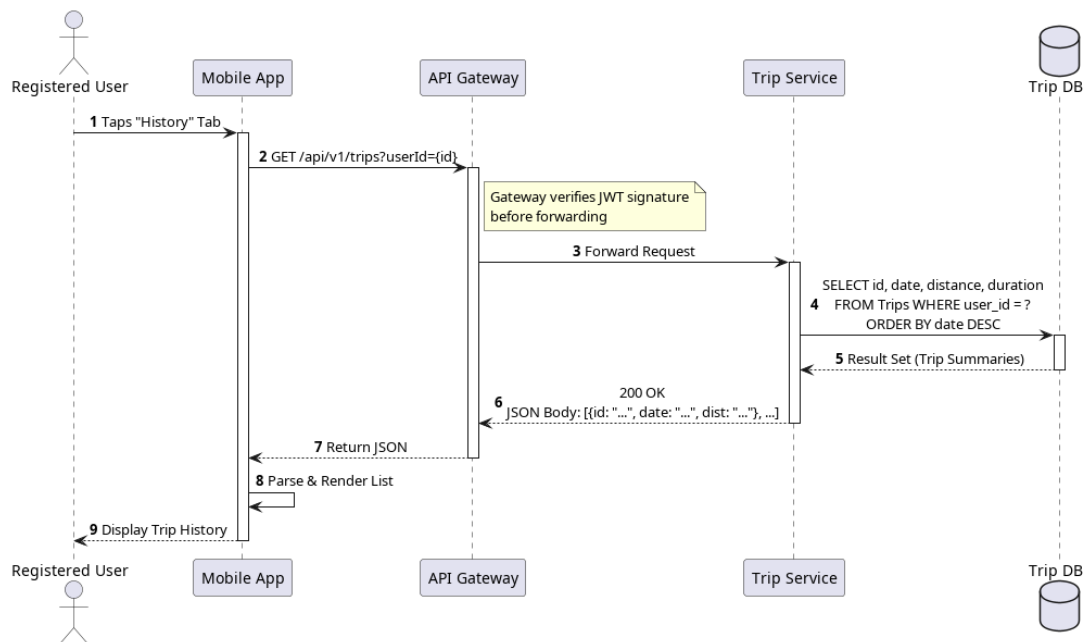


Figure 7: User Views their Bike Trip History Sequence Diagram

When a user views their bike trip history, the mobile application sends a GET request to the API Gateway. The gateway verifies the JWT signature, ensuring the user is properly authenticated, before forwarding the request to the Trip Service, which retrieves data from the Trip DB, which is then sent back to the application.

2.5.6 User views their Bike Trip Statistics

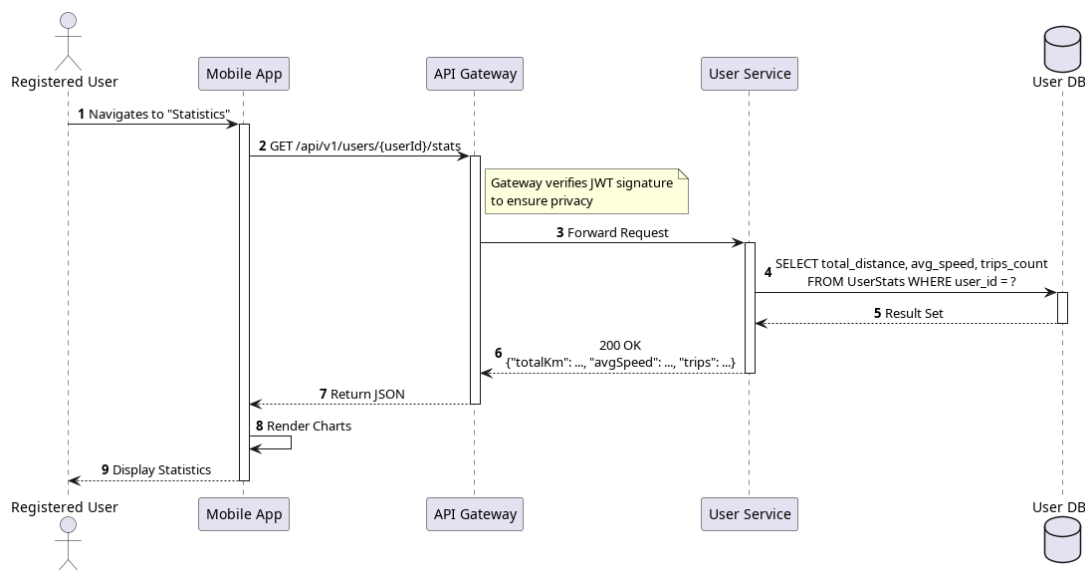


Figure 8: User views their Bike Trip Statistics Sequence Diagram

When a user views their aggregate statistics, the mobile application sends a GET request to the API Gateway. The gateway verifies the JWT signature, ensuring the user is properly authenticated, before

forwarding the request to the User Service, which retrieves data from the User DB, which is then sent back to the application. The statistics data is computed by the Data Processing Service asynchronously each time a trip is added, modified or removed, and stored in the User DB, so as to not perform heavy computations by querying the Trip Service and Trip DB each time a user views their aggregate statistics.

2.5.7 User Publishes Recorded Trip

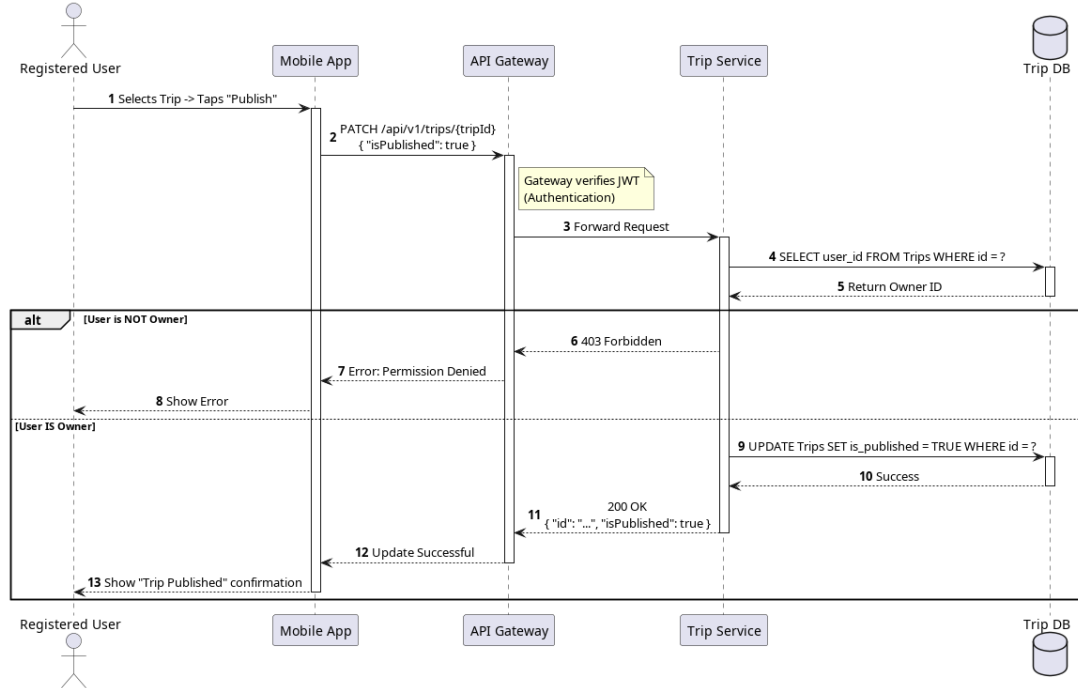


Figure 9: User Publishes Recorded Trip Sequence Diagram

When a user publishes one of their trips, the mobile application sends a PATCH request to the API gateway, modifying the published flag of the trip. The gateway verifies the JWT signature, ensuring the user is properly authenticated, before forwarding the request to the Trip Service, which validates the request and performs the change by querying the Trip DB.

2.5.8 User Modifies one of their Saved Trips

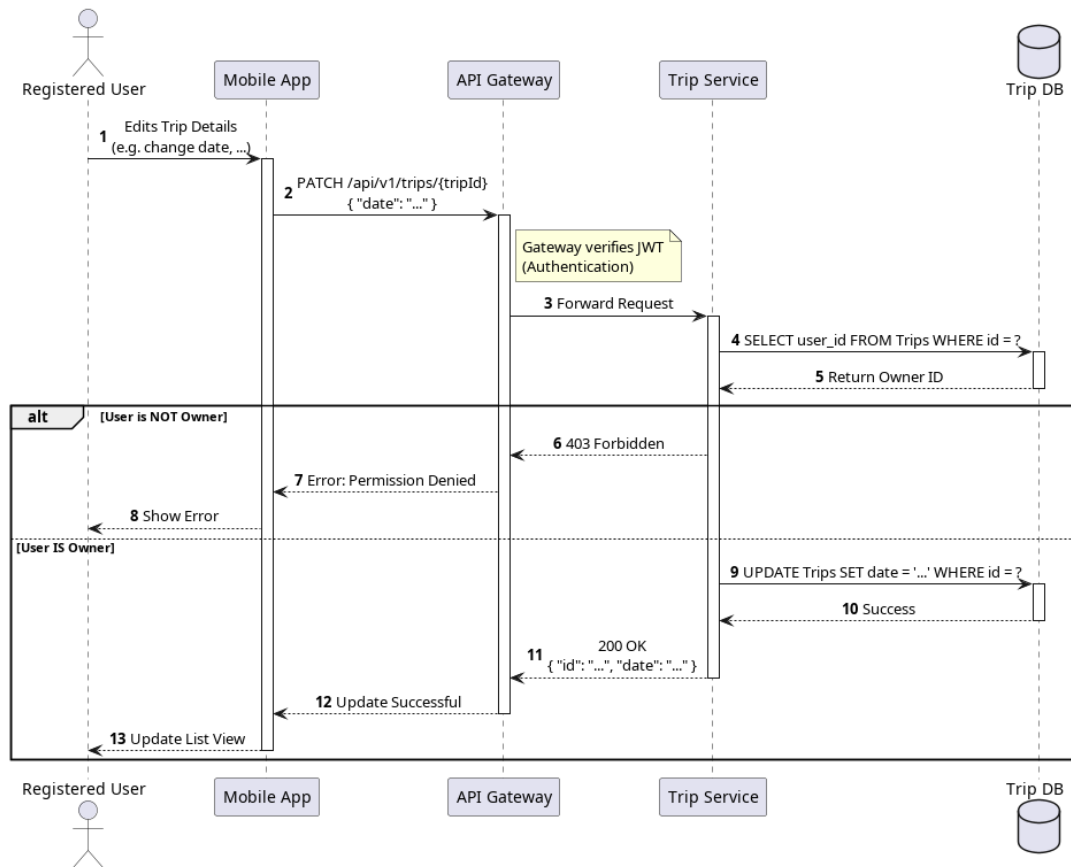


Figure 10: User Modifies one of their Saved Trips Sequence Diagram

When a user modifies one of their saved trips, the mobile application sends a PATCH request to the API gateway, containing the edited data. The gateway verifies the JWT signature, ensuring the user is properly authenticated, before forwarding the request to the Trip Service, which validates the request and performs the change by querying the Trip DB.

2.5.9 User Views Possible Bike Routes On Map

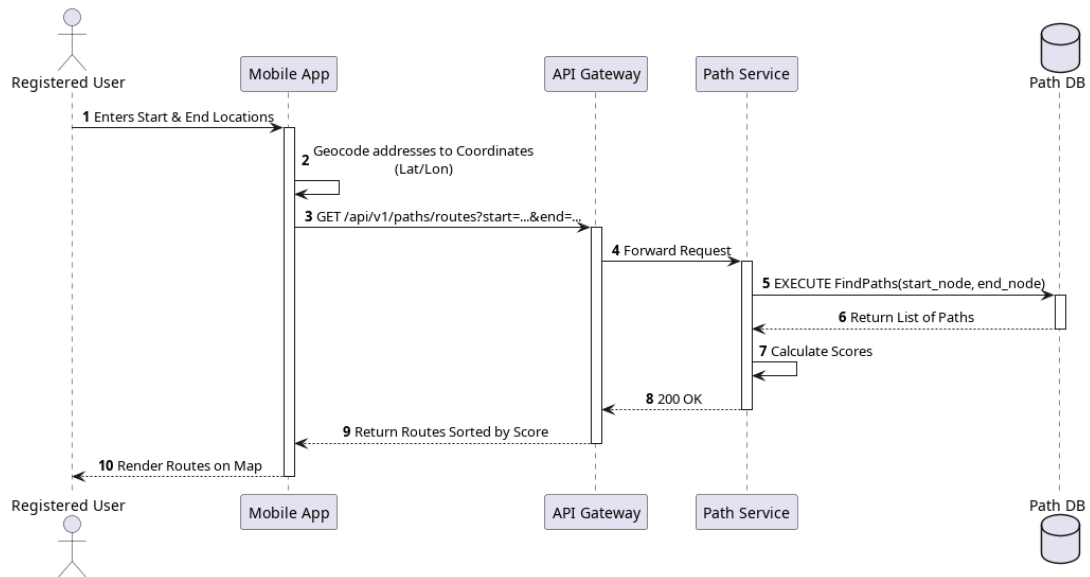


Figure 11: User Views Possible Bike Routes On Map Sequence Diagram

When a user goes to the path finding section of the mobile application, and inserts a starting and ending destination, the app sends a GET request to the API gateway, which then forwards it to the Path Service (no authentication is required for this functionality, so the gateway does not verify the JWT signature). The service then queries the Path DB to find all possible routes between the two points, calculates the scores for each route, and sends them back to the mobile application, which renders them on the map highlighted based on their score.

2.5.10 User Views Additional Information about Selected Path on Map

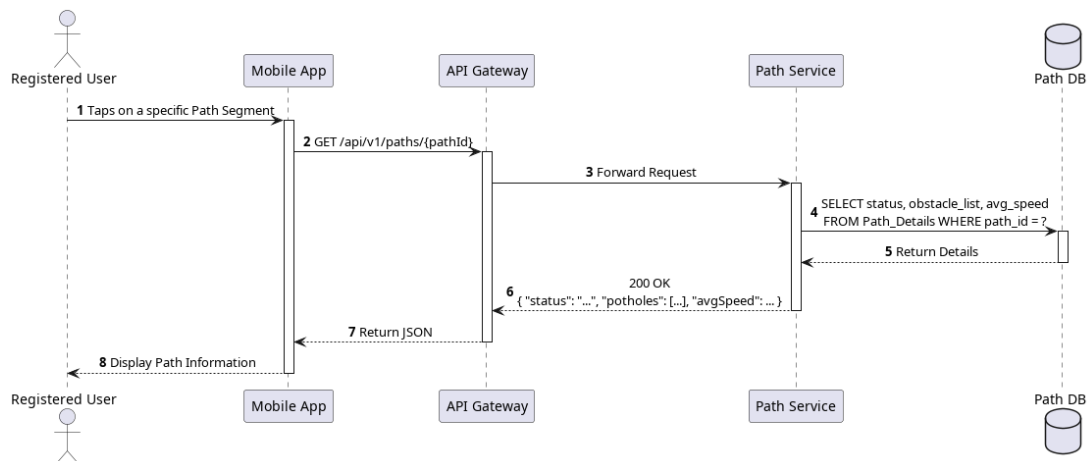


Figure 12: User Views Additional Information about Selected Path on Map Sequence Diagram

When a user selects a specific path segment on the mobile app, to view additional details about it, the application sends a GET request to the API gateway, which then forwards it to the Path Service (no authentication is required for this functionality, so the gateway does not verify the JWT signature). The service retrieves the data from the Path DB, and returns it to the mobile app.

2.5.11 User Starts Navigation on Selected Bike Path

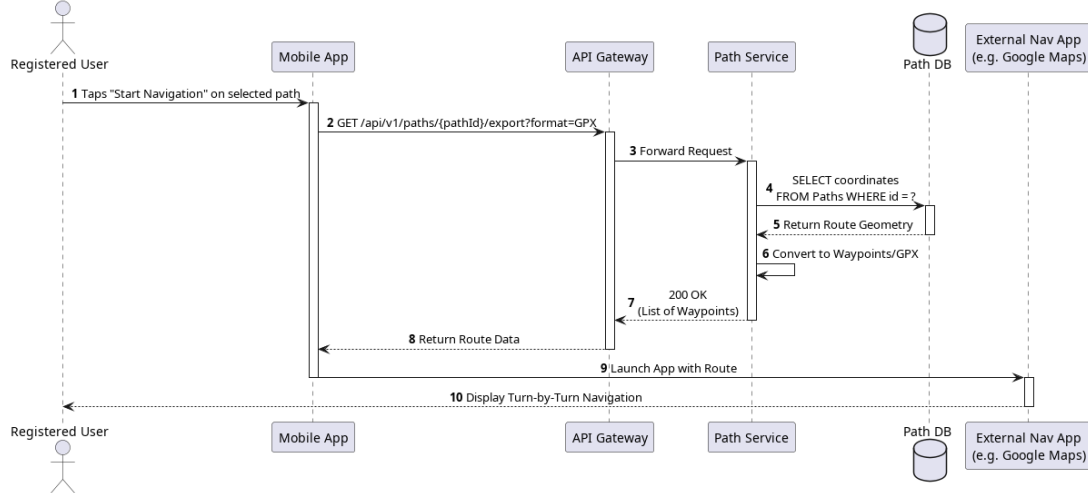


Figure 13: User Starts Navigation on Selected Bike Path Sequence Diagram

When a user starts navigation on one of the identified paths, the mobile app sends a GET request to the API Gateway, which forwards the request to the Path Service. The service retrieves the coordinates of the desired path from the Path DB, and sends them back to the application, which will then launch a third-party navigation application with the selected route.

2.5.12 User Modifies Profile Data

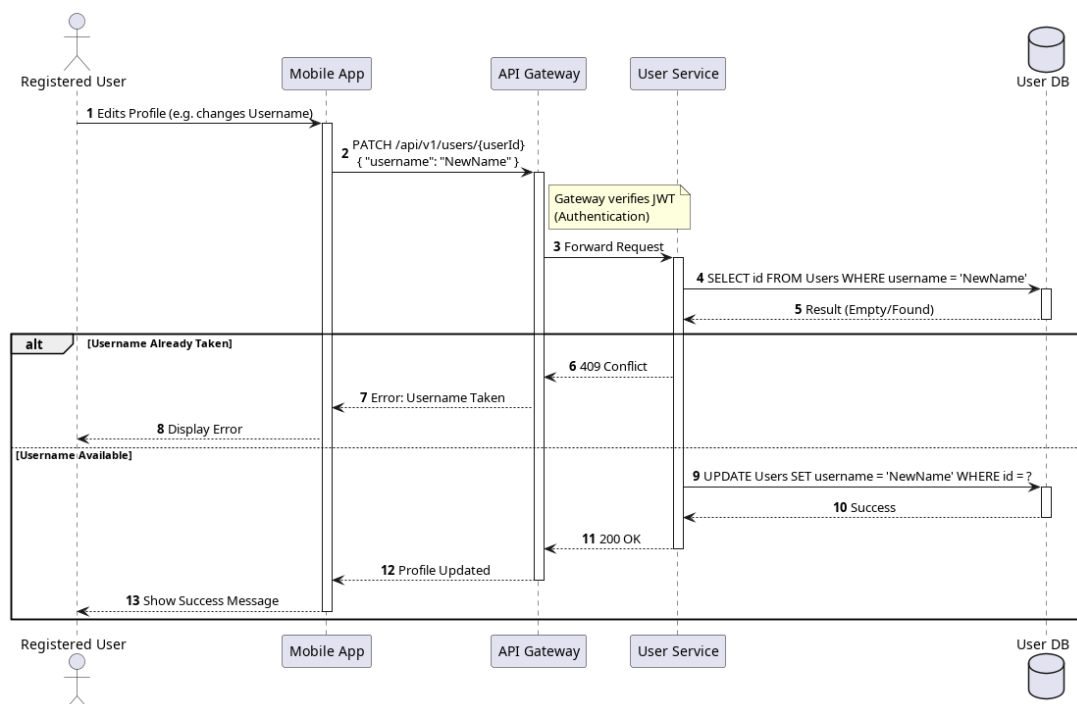


Figure 14: User Modifies Profile Data Sequence Diagram

When a user modifies their profile data, the mobile application sends a PATCH request to the API gateway, with the relevant new data. The gateway verifies the JWT signature, ensuring the user is properly

authenticated, then forwards the request to the User Service, which validates the data and performs the necessary updates to the User DB.

2.5.13 User Logs Out of the Account

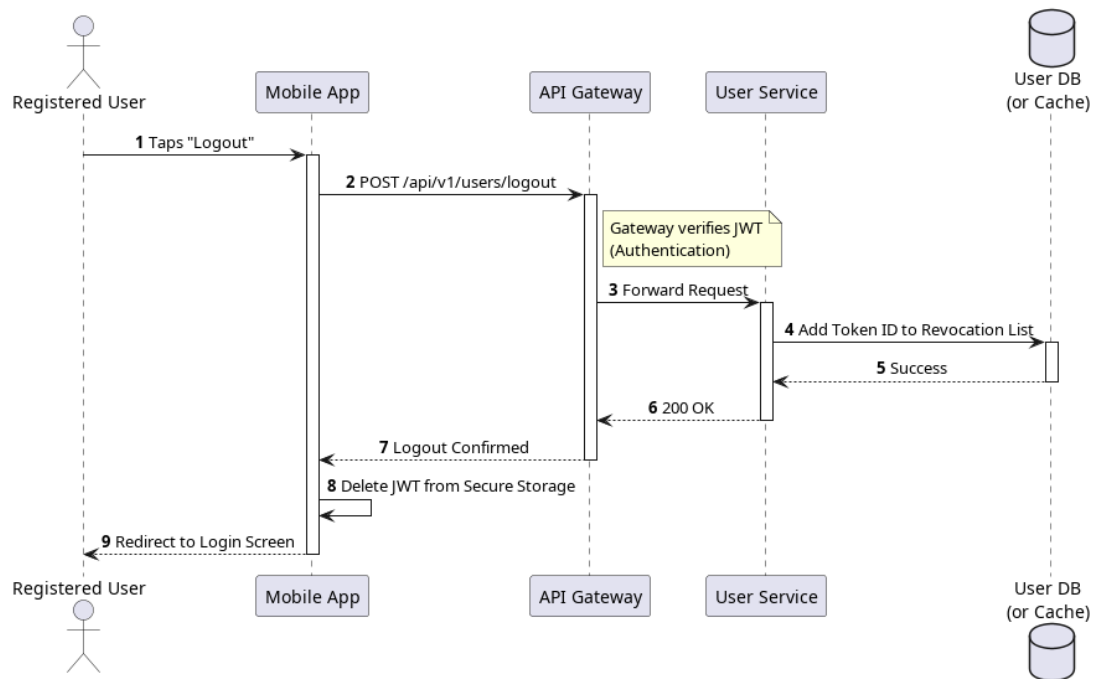


Figure 15: User Logs Out of the Account Sequence Diagram

When a user wishes to log out of their account, the mobile application sends a POST request to the API gateway, which verifies the JWT signature, ensuring the user is properly authenticated, and then forwards the request to the User Service. The service proceeds to add the token id to a revocation list, ensuring that the user is no longer authenticated, and sends confirmation back to the mobile app, which proceeds to delete its JWT from storage.

2.5.14 User Deletes Account

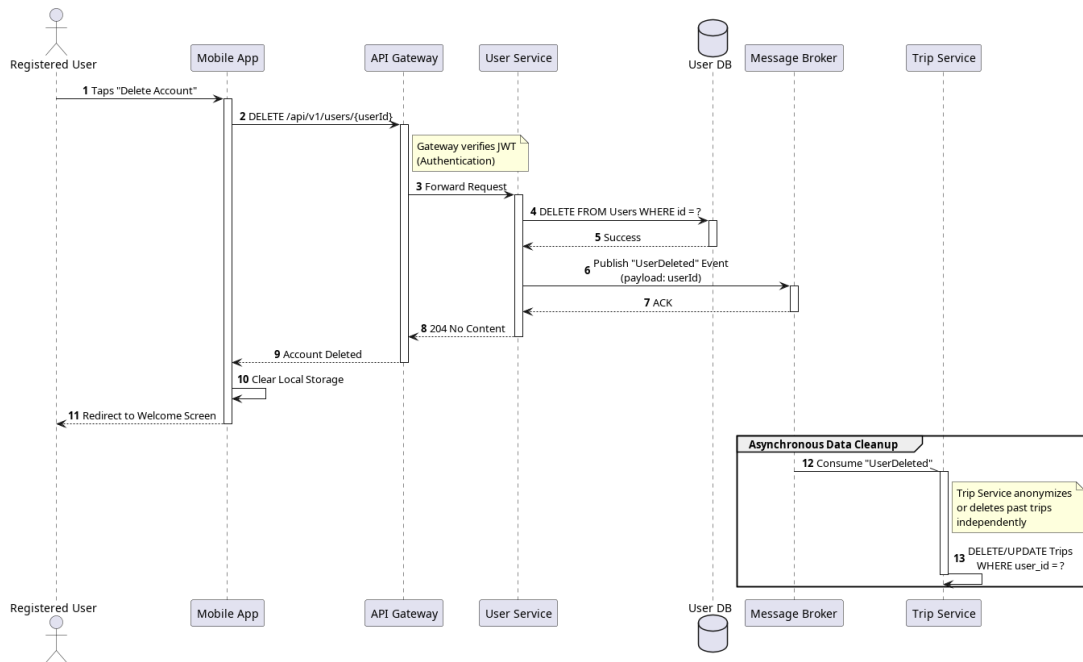


Figure 16: User Deletes Account Sequence Diagram

When a user deletes their account, the mobile application sends a DELETE request to the API gateway, which verifies the JWT signature, ensuring the user is properly authenticated, and then forwards the request to the User Service. The service proceeds to delete all data pertinent to the user from the User DB, and then publishes a 'UserDeleted' event to the Message Broker; this ensures that other services, such as the Trip Service, can asynchronously delete or anonymize any data relative to the user from their DB.

2.5.15 User Recovers Password of the Account

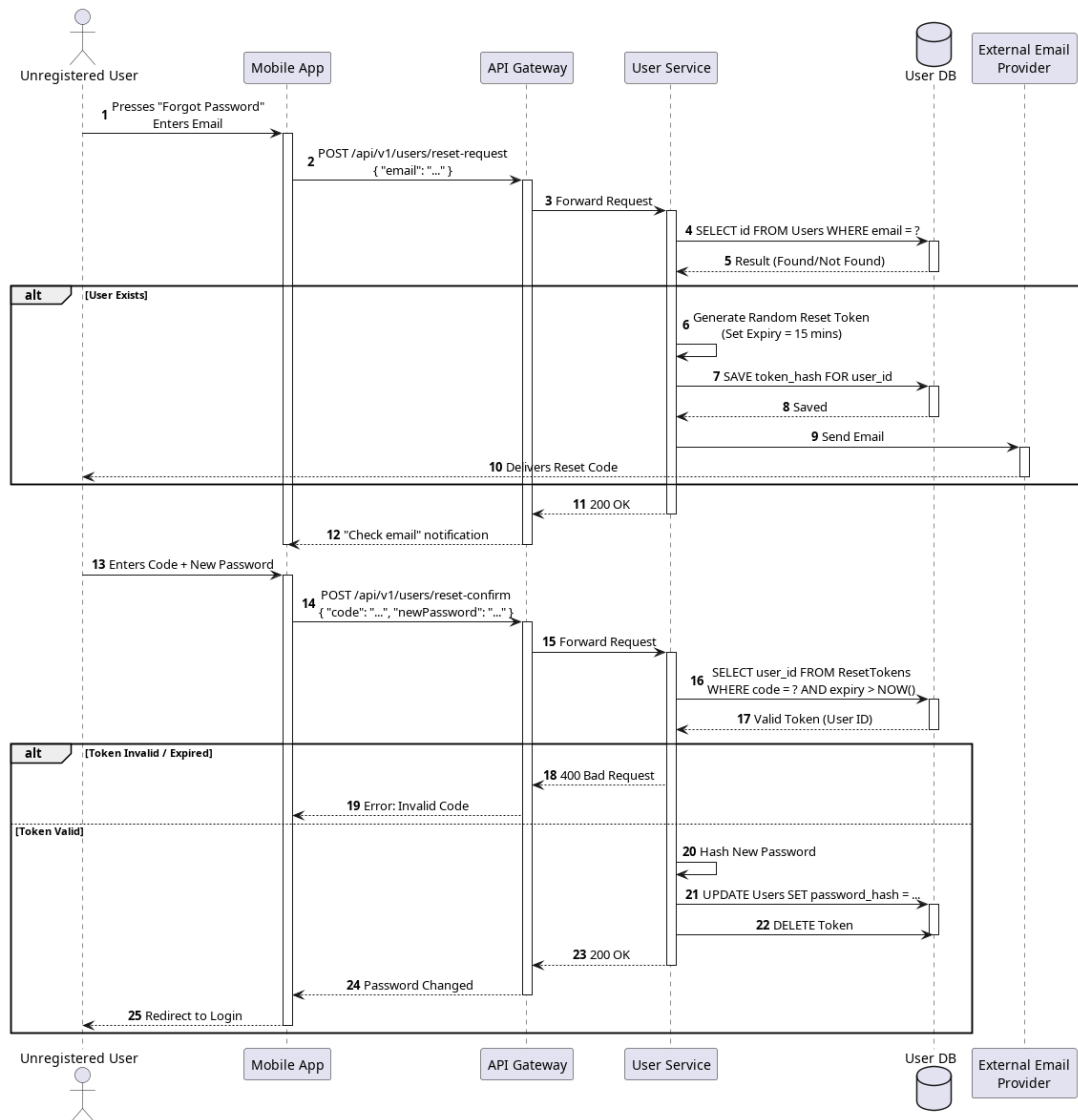


Figure 17: User Recovers Password of the Account Sequence Diagram

When a user wants to recover their password, the mobile application sends a POST request to the API gateway, which forwards the request to the User Service. The service generates a random token and saves it in the User DB, then proceeds to send an email to the address provided by the user, containing the token code. If the user then inserts the code, along with a new password, the app will once again send a POST request to the API gateway, which is forwarded to the User Service. The service will validate the provided token, and in case of success hash and store the new password.

2.5.16 User Deletes a Recorded Trip

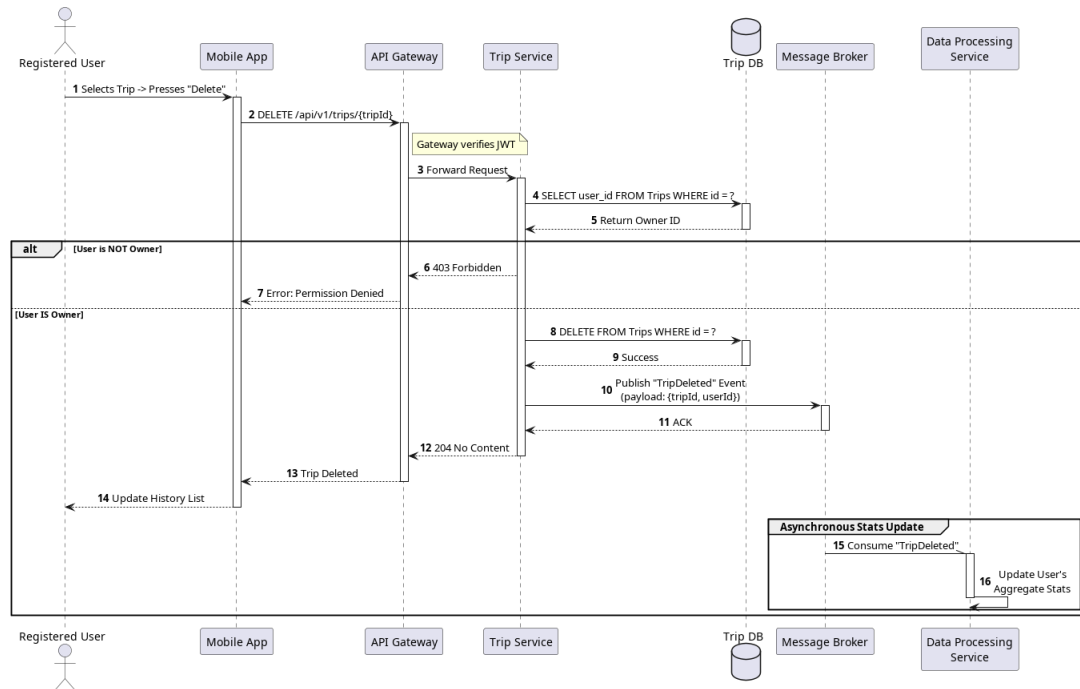


Figure 18: User Deletes a Recorded Trip Sequence Diagram

When a user deletes a trip from their history, the mobile application sends a DELETE request to the API Gateway, which verifies the JWT signature, ensuring the user is properly authenticated, and then forwards it to the Trip Service. The service validates the request, then in case of success proceeds to delete the trip from the Trip DB. The service then publishes a ‘TripDeleted’ event to the Message Broker, so that the Data Processing Service can asynchronously update the shared statistics.

2.6 Selected Architectural Styles and Patterns

Microservices Architecture

The system is divided into a series of lightweight, independent services. Each service runs its own processes and manages its own database, ensuring it is decoupled from the other services. This style was selected to enable independent scalability (e.g. allowing the system to handle a high volume influx of data during the recording of a trip without impacting the other functionalities of the system), and to facilitate deployment by enabling independent deployment of the services.

REST (Representational State Transfer)

The synchronous communication between the Mobile Application and the system, through the API Gateway, follows the REST architectural style. The API is designed around resources, not actions; it uses standard HTTP verbs for operations, and remains stateless through the use of JWTs for authentication. This style was chosen to ensure that the interface would be easily cacheable and easy to use for the mobile client application.

Publish-Subscribe Pattern

To handle the high frequency influx of sensor data during an automated recording, without blocking the user interface, the system employs a Publish-Subscribe pattern. The Trip Service acts as a publisher, sending events to the message broker, while the Data Processing service acts as a subscriber, consuming

the events asynchronously. This decouples the acquisition of data from its computation and processing, allowing the UI to be highly responsive independently from the workload.

Circuit Breaker Pattern

Since cascading failures are a common risk in microservice architectures, the system employs circuit breakers between all of the various services. This allows a service to 'fail fast', ensuring that in case of failure the circuit breaker opens, allowing the rest of the service to continue functioning correctly by invoking fallback methods.

3 User Interface Design

In this section, we describe the user interface of our application. The interface is designed to be intuitive and user-friendly, allowing users to easily navigate through the various features and functionalities of BBP.

Given the nature of BBP, the user interface is user-friendly, providing easy access to all the features and functionalities.

Pages and Navigation

The application is structured into several pages to ensure a logical user flow and an intuitive experience.

The following pages represent the core architecture of the platform:

- **Login and Register Page:** Handles user authentication and registering, allowing users to securely access their accounts or create new ones.
- **Home Page:** Serves as the primary dashboard for users, for inserting and starting registering trips and visualizing recent activities.
- **Search Page:** Enables users to search for published routes on the map with all information about the trip.
- **Your Trips Page:** A central hub for managing personal travel plans, displaying both upcoming itineraries and historical booking data.
- **Account Page:** Dedicated to user profile management, where individuals can update personal information, adjust security settings, and manage application preferences.

Apart from the Login and Register Page, that is the first page you encounter when opening the application the first time, after the login all other pages are accessible via a bottom navigation bar, allowing users to switch between them easily.

3.1 Login and Register Page

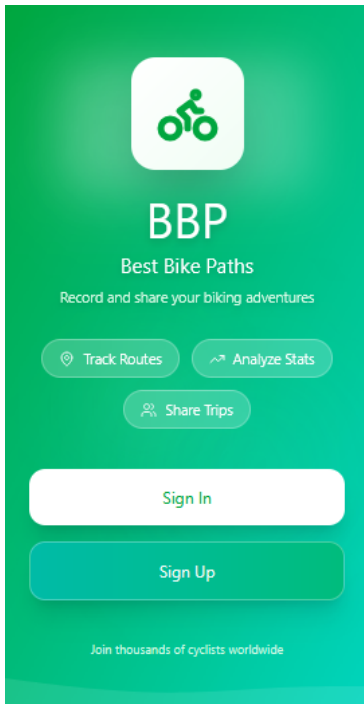


Figure 19: Initial Page

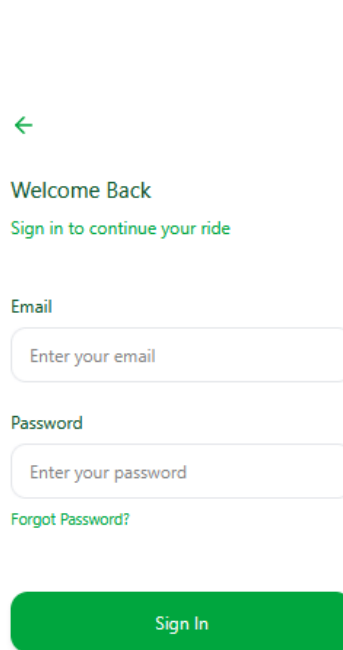


Figure 20: Login Page

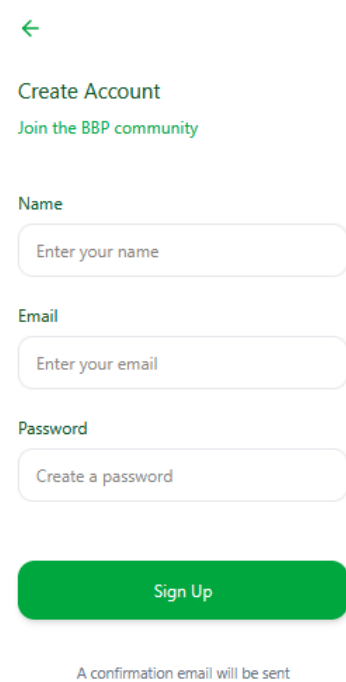


Figure 21: Register Page

The first page when you're not logged in is shown in Figure 19, where you can choose to log in or register a new account.

If you choose to log in, you will be directed to the Login Page shown in Figure 20, where you can enter your credentials to access your account.

If you don't have an account yet, you can click on the "Sign Up" button to be directed to the Register Page shown in Figure 21, where you can create a new account by providing the required information.

3.2 Home Page

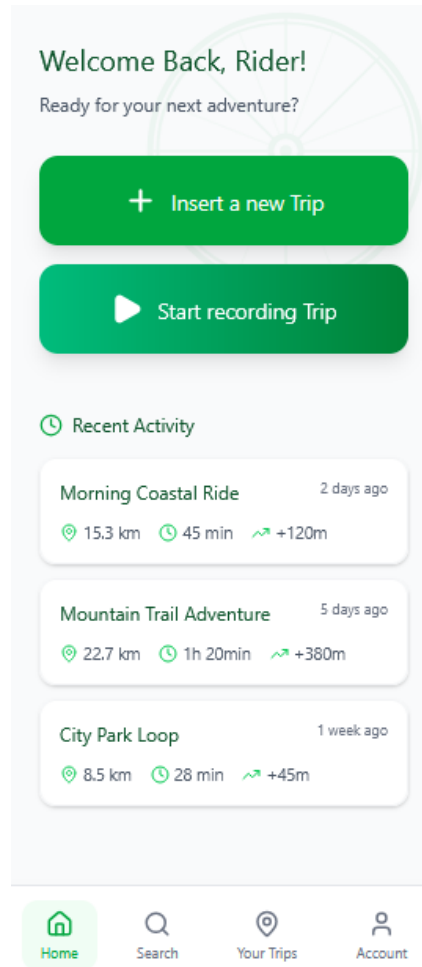


Figure 22: Home Page

The Home Page, shown in Figure 22, serves as the main dashboard for users.

Here, users can manually insert a trip by clicking on the "Insert a new Trip" button, that will open a form that user have to compile to submit the trip, or start the automatic recording of a new trip by clicking on the "Start recording Trip" button, which will guide them through the trip registration process.

Additionally, the Home Page displays recent activities and updates related to the user's account.

3.3 Search Page

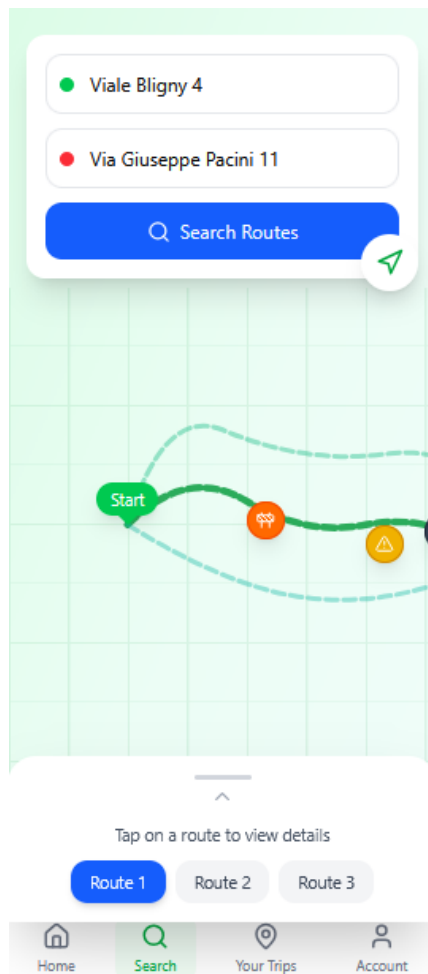


Figure 23: Search Page

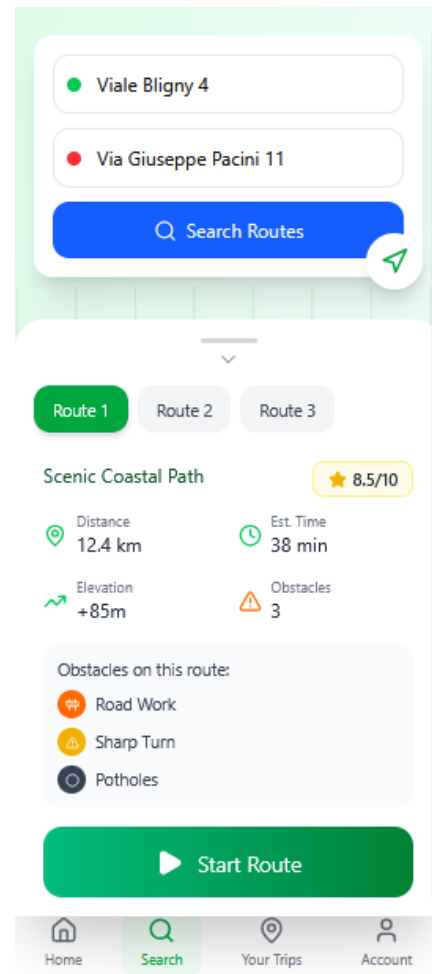


Figure 24: Route Details

The Search Page, shown in Figure 23, allows users to search for published routes on the map inserting start and end point in the forms on the upper part of the page and clicking on them on the map to view detailed information about each trip, as illustrated in Figure 24.

3.4 Your Trips Page

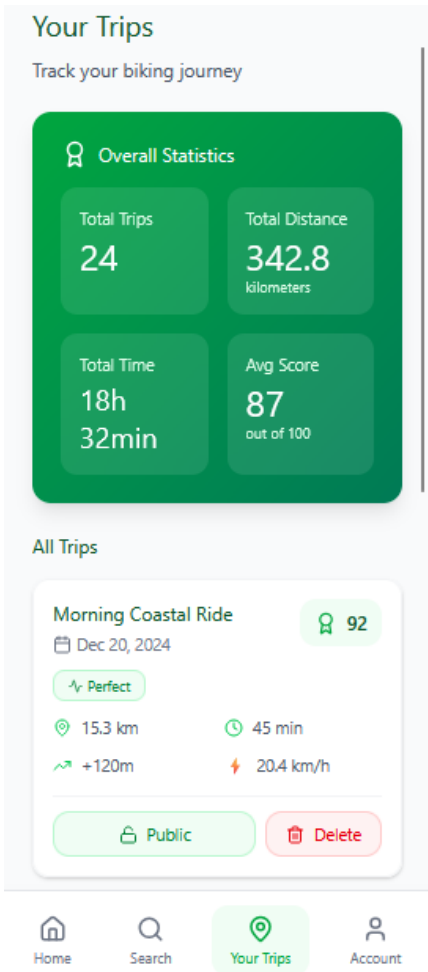


Figure 25: Your Trips Page

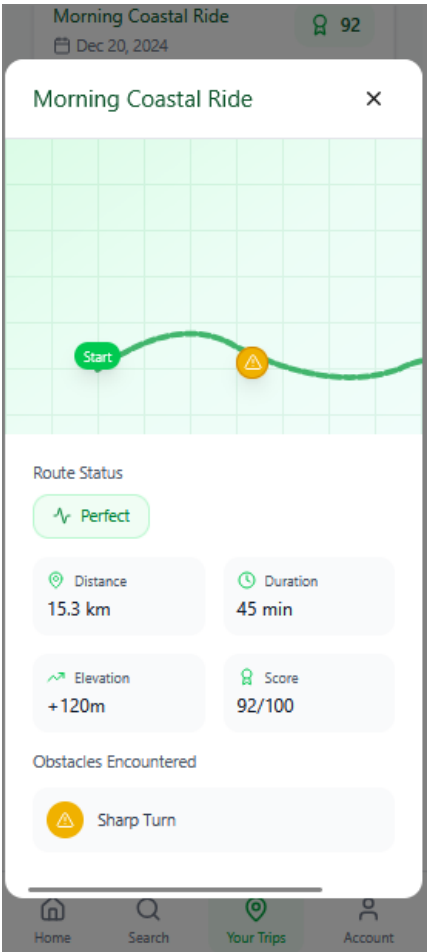


Figure 26: Trip Details

The Your Trips Page, shown in Figure 25, is where users can manage their personal travel plans. It displays both upcoming itineraries and historical booking data. By clicking on a specific trip, users can view detailed information about that trip, as shown in Figure 26.

3.5 Account Page

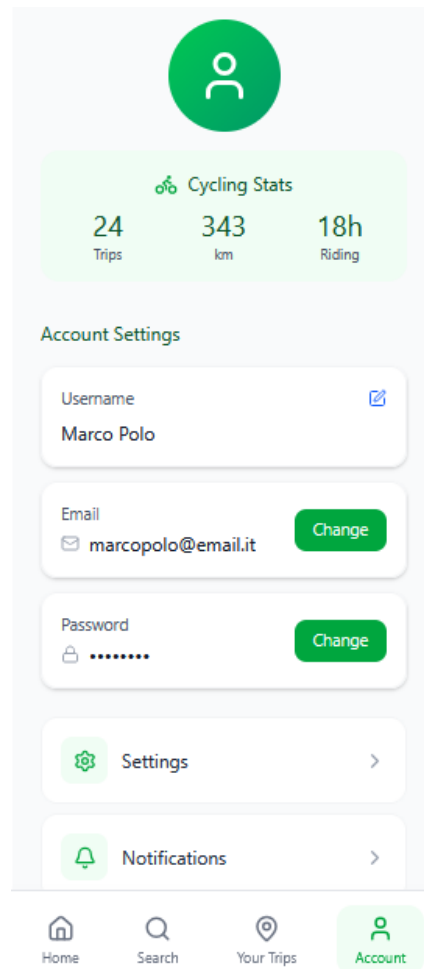


Figure 27: Account Page

The Account Page, shown in Figure 27, is dedicated to user profile management.

Here, users can update their personal information like name, email, and password and manage application preferences according to their needs.

4 Requirements Traceability

The following table lists the traceability between the identified requirements and the corresponding components and modules implemented in the system.

Table 5: Requirements traceability

Component	Requirements
Mobile Application	R1: System accepts account registration data R5: System allows users to login to their account by authenticating via email (or username) and password R6: System allows users to logout of their account R7: System allows users to modify their account information (username and personal data) R8: System allows users to modify/recover their password via email verification R9: System allows users to delete their account, with all the associated data R10: System allows users to start automated trip recording R11: System requests necessary sensor and GPS permissions when automated recording is started R14: System collects GPS coordinates during automated trip recording R15: System reconstructs traveled path from collected GPS coordinates R16: System collects data from mobile device's accelerometer and gyroscope during automated trip recording R18: System allows users to manually stop automated trip recording R19: System allows users to manually record trips by inserting starting and ending locations, names of roads in traversal order, start and end times R20: System allows users to specify road status (optimal, medium, sufficient, requires maintenance) during manual recording R21: System allows users to specify obstacle locations during manual recording R32: System displays a list of user's recorded trips showing name, date, start and end locations R33: System allows users to view detailed trip data for any recorded trip, including route map, statistics and weather data R34: System allows users to manually edit trip data for any of their recorded trips R35: System allows users to delete any of their recorded trips R37: System allows users to make published trips private again R38: System allows registered users to publish their recorded trips, making them publicly available R39: System requires the user to correct and confirm data before publishing, if it was acquired automatically R40: System allows users to add/remove obstacle markers during data correction R41: System allows users to modify road status during data correction R42: System allows all users to view published bike paths between specified start and end points R46: System displays all identified paths on a map, with the highest scoring path highlighted and the others with reduced opacity R49: System allows users to select a displayed path R51: System allows users to export selected path to compatible third-party navigation applications

Table 5: Requirements traceability

Component	Requirements
API Gateway	R1: System accepts account registration data R5: System allows users to login to their account R6: System allows users to logout of their account R7: System allows users to modify their account information R8: System allows users to modify/recover their password R9: System allows users to delete their account, with all the associated data R32: System displays a list of user's recorded trips R33: System allows users to view detailed trip data R34: System allows users to manually edit trip data R35: System allows users to delete any of their recorded trips R37: System allows users to make published trips private again R38: System allows registered users to publish their recorded trips R42: System allows all users to view published bike paths R46: System displays all identified paths on a map R49: System allows users to select a displayed path R51: System allows users to export selected path to compatible third-party applications
Service Discovery	Enables communication between components for all requirements
User Service	R1: System accepts account registration data R2: System rejects account registrations with duplicate emails or usernames R3: System sends a verification email with an OTP to the email address provided during registration R4: System must verify OTP code before confirming registration R5: System allows users to login to their account by authenticating via email (or username) and password R6: System allows users to logout of their account R7: System allows users to modify their account information (username and personal data) R8: System allows users to modify/recover their password via email verification R9: System allows users to delete their account, with all the associated data

Table 5: Requirements traceability

Component	Requirements
Trip Service	R12: System begins automated path and sensor tracking, after recording initiation, only when user has started biking R19: System allows users to manually record trips by inserting starting and ending locations, names of roads in traversal order, start and end times R22: System persistently stores recorded trips R23: System ensures only author of recorded trips can access unpublished trip records R25: System calculates statistics immediately after trip is saved R32: System displays a list of user's recorded trips showing name, date, start and end locations R33: System allows users to view detailed trip data for any recorded trip, including route map, statistics and weather data R34: System allows users to manually edit trip data for any of their recorded trips R35: System allows users to delete any of their recorded trips R36: System propagates modifications and deletions of trips to their published versions, if present R37: System allows users to make published trips private again R38: System allows registered users to publish their recorded trips, making them publicly available
Data Processing Service	R13: System algorithmically detects if a user has started biking from speed data R15: System reconstructs traveled path from collected GPS coordinates R17: System algorithmically interprets collected sensor data to detect potential obstacles along the path R24: System automatically calculates statistics for each recorded trip (distance, average speed, duration, elevation, average slope) R26: System calculates aggregate statistics across all of user's recorded trips R27: System recomputes aggregate statistics upon modification of user's trip record (add/delete/edit trip) R29: System infers road status from detected obstacle density and movement pattern analysis when trip data is recorded automatically R30: System queries an external meteorological service for weather data to enrich trip data after completion

Table 5: Requirements traceability

Component	Requirements
Path Service	R38: System allows registered users to publish their recorded trips, making them publicly available R42: System allows all users to view published bike paths between specified start and end points R43: System identifies all published bike paths that connect specified start and end points R44: System notifies user when no published paths exist between the specified points R45: System calculates a score for each identified bike path based on road status, route effectiveness (distance and directness in reaching destination), amount of obstacles, path difficulty (slope) R47: System displays score values for each path R48: System initially displays a limited amount of the identified paths, providing the option to view all paths R49: System allows users to select a displayed path R50: System allows users to view additional information about selected path (status, obstacles)
Message Broker	R24: System automatically calculates statistics for each recorded trip (distance, average speed, duration, elevation, average slope) R30: System queries an external meteorological service for weather data to enrich trip data after completion R36: System propagates modifications and deletions of trips to their published versions, if present R38: System allows registered users to publish their recorded trips, making them publicly available
Databases & DBMS	R2: System rejects account registrations with duplicate emails or usernames R9: System allows users to delete their account, with all the associated data R22: System persistently stores recorded trips R23: System ensures only author of recorded trips can access unpublished trip records R28: System persistently stores computed statistics with corresponding trip data R31: System stores a notice in trip data when weather data is unavailable R35: System allows users to delete any of their recorded trips R36: System propagates modifications and deletions of trips to their published versions, if present R43: System identifies all published bike paths that connect specified start and end points
Map Provider	R33: System allows users to view detailed trip data for any recorded trip, including route map, statistics and weather data R42: System allows all users to view published bike paths between specified start and end points R46: System displays all identified paths on a map, with the highest scoring path highlighted and the others with reduced opacity R48: System initially displays a limited amount of the identified paths, providing the option to view all paths

Table 5: Requirements traceability

Component	Requirements
Weather Provider	R30: System queries an external meteorological service for weather data to enrich trip data after completion R33: System allows users to view detailed trip data for any recorded trip, including route map, statistics and weather data

5 Implementation, Integration and Test Plan

5.1 Implementation plan

The implementation of BBP will be executed in a phased manner to manage the complexities of a distributed system.

Initially, development will focus on the infrastructure layer using Spring Cloud and Netflix components. This includes setting up the Service Discovery and API Gateway to establish a robust routing mechanism for all incoming client requests.

Once the infrastructure is stable, work will proceed to the User Service to manage registration and authentication processes.

Following this, the Trip Service and Path Service will be developed to handle the core logic of recording cycling activities and computing path scores based on user-submitted status reports.

Finally, the Data Processing Service will be implemented to consume events from the Kafka broker, allowing for the generation of performance statistics and the integration of external meteorological data.

5.2 Integration plan

The integration strategy for BBP combines synchronous and asynchronous patterns to balance performance with reliability. Direct interactions, such as user logins or path searches, will be handled synchronously through the API Gateway via RESTful calls, given the Microservices architecture. This ensures immediate feedback for high-priority user actions.

Instead more complex operations will utilize an event-driven approach. When the Trip Service completes a recording, it will publish a "Trip Recorded" event to the Kafka message broker. This allows the Data Processing Service to aggregate statistics and the Path Service to update the path database without blocking the user's mobile application.

Furthermore, external integrations with Map and Weather providers will be encapsulated within their respective services to maintain the principle of loose coupling of Microservices.

5.3 Test plan

The testing phase ensures that all functional requirements are met and that the system remains stable under load in each development stage and for each microservice within the architecture.

- **Unit Testing:** Each one of the microservices will undergo rigorous unit testing using JUnit to validate business logic in isolation.
- **Integration Testing:** We will verify the exchange of messages through a tool like Kafka and ensure that the three independent databases (User, Trip, and Path) maintain data integrity during each type of operations.
- **Functional and System Testing** This will focus on critical BBP scenarios, such as the accuracy of path score calculations and the reliability of the automated mode in reconstructing GPS paths. We will also simulate concurrent user access to ensure the system can handle the expected community load.
- **User Interface Testing:** The mobile application will be tested on various devices and operating systems to ensure a consistent user experience and responsiveness and ease of use from client perspective.
- **Security Testing:** We will conduct vulnerability assessments and penetration testing to safeguard user data, particularly focusing on authentication and authorization mechanisms, utilizing notorious hacking strategies.

- **Performance Testing:** Load testing will be conducted to assess system performance under peak conditions, ensuring that response time remain acceptable.

6 Effort Spent

Member of group	Effort spent	
Bedini Simone	Introduction	1 <i>h</i>
	Architectural design	25 <i>h</i>
	User interface design	3 <i>h</i>
	Requirements Traceability	5 <i>h</i>
	Implementation, Integration and Test Plan	3 <i>h</i>
Cantoli Angelo	Introduction	4 <i>h</i>
	Architectural design	5 <i>h</i>
	User interface design	15 <i>h</i>
	Requirements Traceability	5 <i>h</i>
	Implementation, Integration and Test Plan	15 <i>h</i>

7 References

- Requirements analysis and specification document (RASD) of Best Bike Paths application
- Lecture slides and notes from the course Software Engineering 2 2025/2026

7.1 Used Tools

The following tools have been used to create this document:

- **LaTeX**: for document preparation and typesetting.
- **PlantUML**: for creating UML sequence diagrams included in the document.
- **GitHub**: for version control and collaboration on the document source files.
- **Figma**: for designing user interfaces and prototyping.
- **VSCode**: as the main text editor for writing LaTeX code.
- **StarUML**: for creating the component and deployment diagrams included in the document.

7.2 Use of Generative AI

For the development of the mockups used in the 'User Interface Design' section, **Uizard** was used. This is a generative AI tool that assists in quickly creating user interfaces. We felt this tool was appropriate for the scope of the project, as it simply aided in creating a visual aid for the document, and did not contribute in any way to the textual content and descriptions of the 'User Interface Design' section.